Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07 Информационные системы и программирование

ПРАКТИЧЕСКАЯ РАБОТА № 7

Дисциплина(модуль)/предмет МДК 02.02 Инструментальные средства разработки и отладки программного обеспечения (ПМ.02 Осуществление интеграции программных модулей)

Тема: Разработка диаграммы модулей и структуры проекта для АИС

ВАРИАНТ № 10

Студент (ка)/учащийся: Иванов Егор Михайлович / _____ /
Ф. И. О. подпись

Группа / Класс И14-23-9

Руководитель: Чудинова А.А. /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «24» октября 2026 г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

Введение....3
1 Выделение модулей....4
2 Построение схемы модулей....5
2.1 UML Component Diagram....5
2.2 Диаграмма C4 Model, уровень 2 (Containers)....5
3 Проектирование структуры каталогов....7
Заключение....10
Список использованных источников....11
Приложение А. Исходные тексты диаграмм PlantUML....12

ВВЕДЕНИЕ

Цель работы — для выбранного ранее варианта АИС спроектировать модульную структуру клиентского приложения и интеграционного шлюза: выделить ключевые модули, изобразить их связи на диаграмме модулей (UML Component Diagram и C4 Model) и отобразить модульную структуру в виде дерева каталогов решения Visual Studio.

Вариант — тот же, что в практической работе № 6 (номер в списке группы 10, каталог из 15 вариантов): **АИС «Спортивный комплекс и секции»** с интеграционным шлюзом записи в секции, тренажёрный зал и аренду инвентаря (эндпоинты `GET /sections/schedule` и `POST /equipment/rent`).

Пошаговый план практики (по заданию темы 2.2 № 10):

1) **Выделение модулей** — определить не менее 3–4 ключевых модулей системы (UI.Desktop, Core.BusinessLogic, Data.ApiClient, Security.Auth и др.).
2) **Построение схемы** — нарисовать диаграмму модулей в нотациях UML Component Diagram и C4 Model со стрелками зависимостей (инструмент — PlantUML).
3) **Проектирование структуры каталогов** — отобразить модульную структуру в виде дерева папок решения Visual Studio.

Использованное программное обеспечение: PlantUML (текстовое описание диаграмм в файлах .puml); Microsoft Visual Studio Community (версия 18); .NET SDK 8.0; xUnit 2.9.3. Решение создано и собрано на компьютере студента, скриншоты сделаны инструментом Windows «Ножницы».

1 ВЫДЕЛЕНИЕ МОДУЛЕЙ

Система разделена на шесть модулей (проектов .NET) и один тестовый проект. Каждый модуль отвечает за одну область, а зависимости направлены от интерфейса к ядру и к общим контрактам (таблица 1).

Таблица 1 – Модули АИС «Спортивный комплекс и секции»

Модуль (проект)	Тип	Ответственность	Зависит от
SportsComplex.UI.Desktop	WinForms-приложение (net8.0-windows)	Интерфейс студента и тренера: форма расписания и бронирования; Composition Root — сборка модулей в Program.cs	Core.BusinessLogic, Data.ApiClient, Security.Auth
SportsComplex.Core.BusinessLogic	Библиотека классов	Бизнес-логика: ScheduleService, RentalService, правила аренды RentalPolicy (≤ 3 ед. одной позиции, бронь закрывается за 30 мин); порт ISportsGatewayClient	Shared.Contracts
SportsComplex.Data.ApiClient	Библиотека классов	Адаптер к шлюзу: SportsGatewayHttpClient реализует ISportsGatewayClient через HttpClient и REST/JSON	Core.BusinessLogic, Security.Auth, Shared.Contracts
SportsComplex.Security.Auth	Библиотека классов	Аутентификация: ITokenProvider, хранение JWT, BearerTokenHandler добавляет заголовок Authorization	—
SportsComplex.Shared.Contracts	Библиотека классов	Общие DTO по контракту openapi.yaml (SectionSessionDto, RentItemDto …) и маршруты ApiRoutes	—
SportsComplex.Gateway.Api	ASP.NET Core Web API	Интеграционный шлюз: GET /v1/sections/schedule, POST /v1/equipment/rent	Shared.Contracts
SportsComplex.Core.Tests	xUnit-тесты	Модульные тесты правил RentalPolicy	Core.BusinessLogic

Ключевое архитектурное решение — **инверсия зависимостей**: интерфейс `ISportsGatewayClient` объявлен в ядре (Core.BusinessLogic), а реализован в модуле Data.ApiClient. Благодаря этому бизнес-логика не знает про HTTP и тестируется без сети, а модуль доступа к данным можно заменить (например, на локальный кэш) без изменения ядра. Циклических зависимостей между модулями нет.

2 ПОСТРОЕНИЕ СХЕМЫ МОДУЛЕЙ

2.1 UML Component Diagram

Диаграмма компонентов (рисунок 1) описана на языке PlantUML в файле `docs/diagrams/components.puml` (листинг А.1). Компоненты сгруппированы в пакеты «Клиентское приложение (Windows, .NET 8)», «Сервер» и общий модуль контрактов. Пунктирные стрелки — зависимости на уровне кода (ссылки между проектами), сплошные — взаимодействие во время работы: клиент вызывает шлюз по HTTPS + JSON согласно контракту openapi.yaml, шлюз работает с БД спорткомплекса, модуль аутентификации получает JWT у SSO вуза по OAuth 2.0.

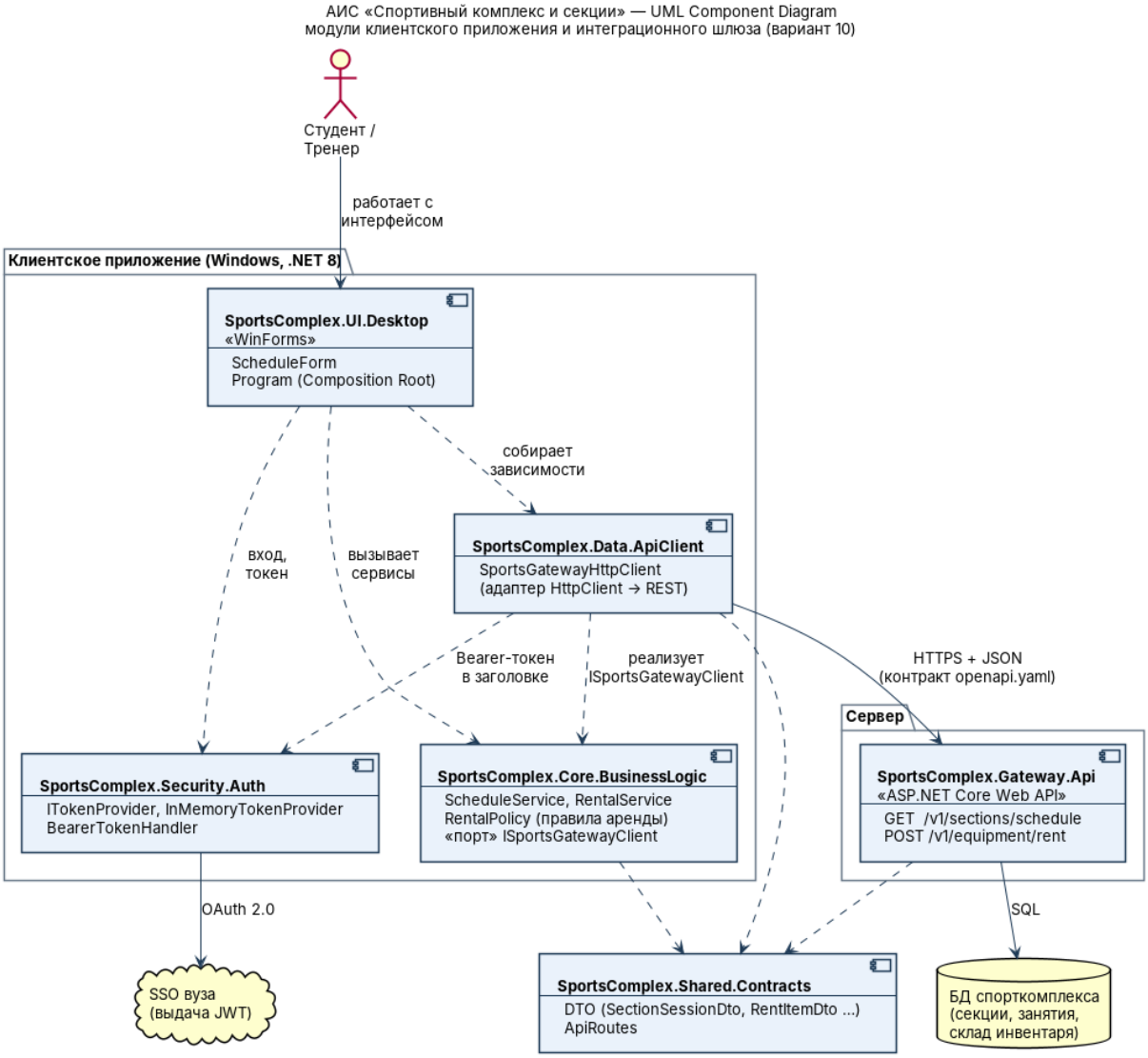


Рисунок 1 – UML Component Diagram модулей клиентского приложения и шлюза

2.2 Диаграмма C4 Model, уровень 2 (Containers)

Диаграмма C4 второго уровня (рисунок 2, файл `docs/diagrams/c4-containers.puml`, листинг А.2) показывает систему крупнее — на уровне

развёртываемых контейнеров: клиентское приложение, интеграционный шлюз и базу данных внутри границы АИС, а также пользователей (студент, тренер) и внешние системы — SSO вуза и СКУД кампуса. Цвета соответствуют нотации C4: тёмно-синий — люди, синий — контейнеры системы, серый — внешние системы.

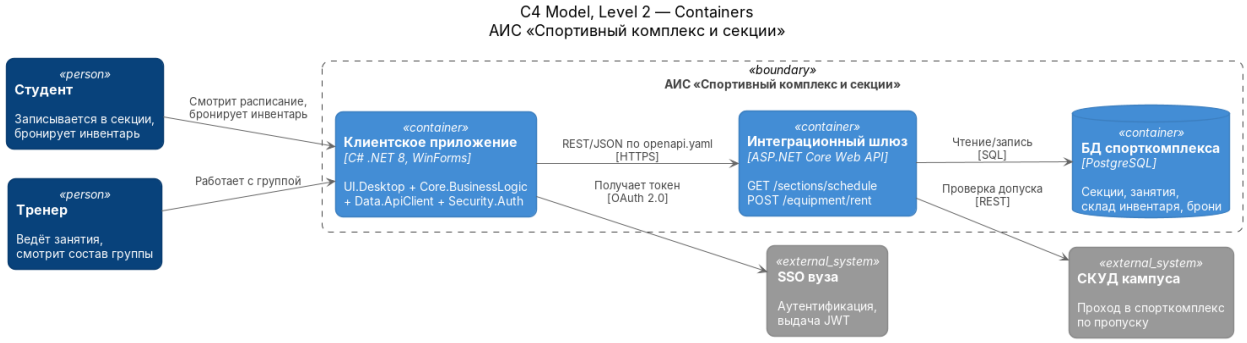


Рисунок 2 – C4 Model, Level 2 — Containers: АИС «Спортивный комплекс и секции»

Обе диаграммы хранятся как текст рядом с кодом (папка docs решения): их можно рецензировать в Pull Request, сравнивать по версиям в Git и перегенерировать при изменении архитектуры.

3 ПРОЕКТИРОВАНИЕ СТРУКТУРЫ КАТАЛОГОВ

Модульная структура перенесена в решение Visual Studio `SportsComplex.AIS.sln`: исходный код модулей — в папке `src`, тесты — в `tests`, архитектурные диаграммы — в `docs/diagrams`. Каждый модуль — отдельный проект, имя проекта совпадает с именем каталога и корневым пространством имён. Дерево каталогов приведено в листинге 1.

Листинг 1 – Дерево каталогов решения SportsComplex.AIS

```
PR7_SportsComplex/
├── docs/
│   └── diagrams/
│       ├── c4-containers.puml
│       └── components.puml
├── src/
│   ├── SportsComplex.Core.BusinessLogic/
│   │   ├── Abstractions/
│   │   │   └── ISportsGatewayClient.cs
│   │   ├── Rules/
│   │   │   └── RentalPolicy.cs
│   │   ├── Services/
│   │   │   ├── RentalService.cs
│   │   │   └── ScheduleService.cs
│   │   └── SportsComplex.Core.BusinessLogic.csproj
│   ├── SportsComplex.Data.ApiClient/
│   │   ├── SportsComplex.Data.ApiClient.csproj
│   │   └── SportsGatewayHttpClient.cs
│   ├── SportsComplex.Gateway.Api/
│   │   ├── Program.cs
│   │   └── SportsComplex.Gateway.Api.csproj
│   ├── SportsComplex.Security.Auth/
│   │   ├── BearerTokenHandler.cs
│   │   ├── InMemoryTokenProvider.cs
│   │   ├── ITokenProvider.cs
│   │   └── SportsComplex.Security.Auth.csproj
│   ├── SportsComplex.Shared.Contracts/
│   │   ├── Dto/
│   │   │   ├── EquipmentRentRequestDto.cs
│   │   │   ├── EquipmentRentResponseDto.cs
│   │   │   ├── RentItemDto.cs
│   │   │   └── SectionSessionDto.cs
│   │   ├── ApiRoutes.cs
│   │   └── SportsComplex.Shared.Contracts.csproj
│   └── SportsComplex.UI.Desktop/
│       ├── Forms/
│       │   └── ScheduleForm.cs
│       ├── Program.cs
│       └── SportsComplex.UI.Desktop.csproj
├── tests/
│   └── SportsComplex.Core.Tests/
│       ├── RentalPolicyTests.cs
│       └── SportsComplex.Core.Tests.csproj
└── SportsComplex.AIS.sln
```

Решение открыто в Visual Studio и пересобрано командой «Сборка → Пересобрать решение»: «Перестроить все: успешно — 7, неудачно — 0», то есть все ссылки между модулями корректны и соответствуют диаграмме. Модульные тесты RentalPolicyTests (проверка правил аренды) пройдены — 5 из 5 (рисунок 3).

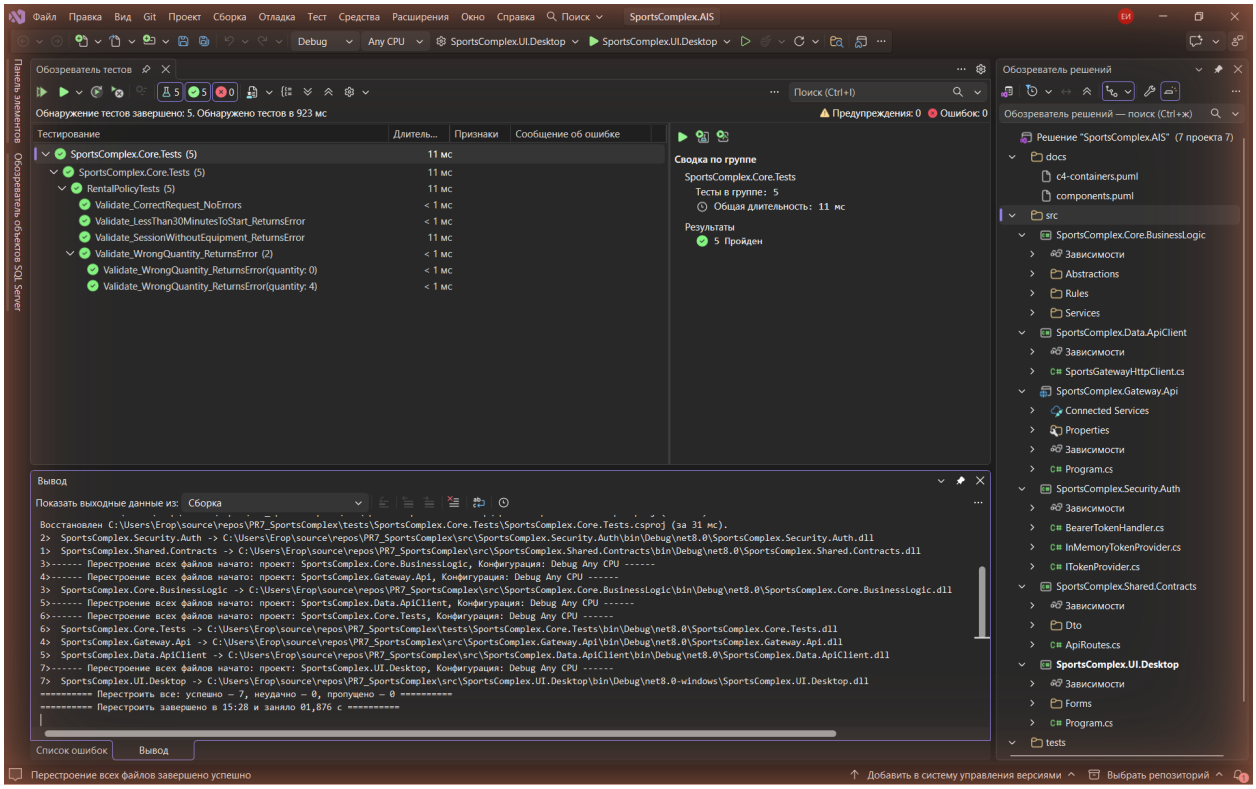


Рисунок 3 – Решение SportsComplex.AIS в Visual Studio: модули в Обозревателе решений, успешная пересборка 7 проектов и 5 пройденных тестов

На рисунке 4 показан исходный текст диаграммы компонентов `components.puml`, который хранится в папке docs решения.

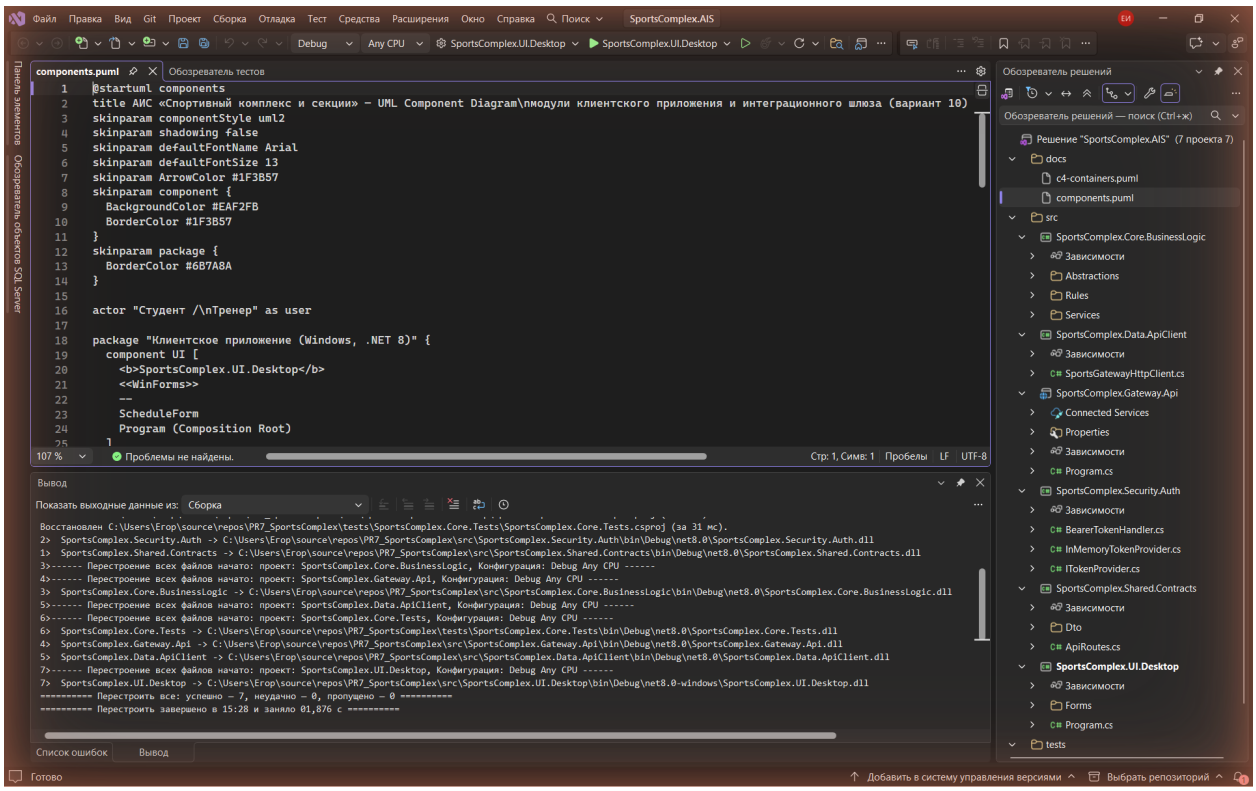


Рисунок 4 – Файл диаграммы components.puml в составе решения

ЗАКЛЮЧЕНИЕ

В работе для АИС «Спортивный комплекс и секции» выделено шесть модулей — UI.Desktop, Core.BusinessLogic, Data.ApiClient, Security.Auth, Shared.Contracts и Gateway.Api — и тестовый проект Core.Tests. Связи между ними изображены на UML Component Diagram и на диаграмме C4 уровня Containers (PlantUML). Модульная структура реализована в виде решения Visual Studio с деревом каталогов src / tests / docs; решение собирается без ошибок, тесты проходят.

Разделение на модули с однонаправленными зависимостями и инверсией зависимостей (порт ISportsGatewayClient в ядре) упрощает параллельную разработку и тестирование, локализует изменения — замена протокола шлюза затрагивает только Data.ApiClient, — и делает архитектуру наглядной для команды и преподавателя благодаря диаграммам, хранящимся вместе с кодом.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1) PlantUML : Component Diagram / PlantUML.com. — URL: https://plantuml.com/component-diagram (дата обращения: 24.10.2026).
2) Brown S. The C4 model for visualising software architecture. — URL: https://c4model.com (дата обращения: 24.10.2026).
3) C4 Model Tool / Visual Paradigm Online. — URL: https://online.visual-paradigm.com/ru/diagrams/features/c4-model-tool/ (дата обращения: 24.10.2026).
4) Solutions and projects in Visual Studio / Microsoft Learn. — URL: https://learn.microsoft.com/visualstudio/ide/solutions-and-projects-in-visual-studio (дата обращения: 24.10.2026).
5) Common web application architectures (Clean Architecture) / Microsoft Learn. — URL: https://learn.microsoft.com/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures (дата обращения: 24.10.2026).

ПРИЛОЖЕНИЕ А. ИСХОДНЫЕ ТЕКСТЫ ДИАГРАММ PLANTUML

Листинг 2 – А.1 — docs/diagrams/components.puml

```
@startuml components
title АИС «Спортивный комплекс и секции» — UML Component Diagram\nмодули клиентского приложения и интеграционного шлюза (вариант 10)
skinparam componentStyle uml2
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 13
skinparam ArrowColor #1F3B57
skinparam component {
  BackgroundColor #EAF2FB
  BorderColor #1F3B57
}
skinparam package {
  BorderColor #6B7A8A
}

actor "Студент /\nТренер" as user

package "Клиентское приложение (Windows, .NET 8)" {
  component UI [
    <b>SportsComplex.UI.Desktop</b>
    <<WinForms>>
    --
    ScheduleForm
    Program (Composition Root)
  ]
  component Core [
    <b>SportsComplex.Core.BusinessLogic</b>
    --
    ScheduleService, RentalService
    RentalPolicy (правила аренды)
    «порт» ISportsGatewayClient
  ]
  component Data [
    <b>SportsComplex.Data.ApiClient</b>
    --
    SportsGatewayHttpClient
    (адаптер HttpClient → REST)
  ]
  component Auth [
    <b>SportsComplex.Security.Auth</b>
    --
    ITokenProvider, InMemoryTokenProvider
    BearerTokenHandler
  ]
}

component Contracts [
  <b>SportsComplex.Shared.Contracts</b>
  --
  DTO (SectionSessionDto, RentItemDto …)
  ApiRoutes
]

package "Сервер" {
  component Gateway [
```

```
    <b>SportsComplex.Gateway.Api</b>
    <<ASP.NET Core Web API>>
    --
    GET  /v1/sections/schedule
    POST /v1/equipment/rent
  ]
}

database "БД спорткомплекса\n(секции, занятия,\nсклад инвентаря)" as DB
cloud "SSO вуза\n(выдача JWT)" as SSO

user --> UI : работает с\nинтерфейсом
UI ..> Core : вызывает\nсервисы
UI ..> Auth : вход,\nтокен
UI ..> Data : собирает\nзависимости
Data ..> Core : реализует\nISportsGatewayClient
Data ..> Auth : Bearer-токен\nв заголовке
Core ..> Contracts
Data ..> Contracts
Gateway ..> Contracts
Data --> Gateway : HTTPS + JSON\n(контракт openapi.yaml)
Gateway --> DB : SQL
Auth --> SSO : OAuth 2.0
@enduml
```

Листинг 3 – А.2 — docs/diagrams/c4-containers.puml

```
@startuml c4-containers
title C4 Model, Level 2 — Containers\nАИС «Спортивный комплекс и секции»
skinparam defaultFontName Arial
skinparam shadowing false
skinparam rectangle {
  RoundCorner 18
  FontColor white
  StereotypeFontColor white
  BorderColor #0B4884
}
skinparam rectangle<<person>> {
  BackgroundColor #08427B
  BorderColor #073B6F
}
skinparam rectangle<<container>> {
  BackgroundColor #438DD5
  BorderColor #3C7FC0
}
skinparam database<<container>> {
  BackgroundColor #438DD5
  BorderColor #3C7FC0
  FontColor white
  StereotypeFontColor white
}
skinparam rectangle<<external_system>> {
  BackgroundColor #999999
  BorderColor #8A8A8A
}
skinparam rectangle<<boundary>> {
  BackgroundColor transparent
  BorderColor #444444
  BorderStyle dashed
  FontColor #444444
```

```
  StereotypeFontColor #444444
}
skinparam ArrowColor #666666
skinparam ArrowFontColor #444444
left to right direction

rectangle "==Студент\n\nЗаписывается в секции,\nбронирует инвентарь" as student <<person>>
rectangle "==Тренер\n\nВедёт занятия,\nсмотрит состав группы" as coach <<person>>

rectangle "АИС «Спортивный комплекс и секции»" as ais <<boundary>> {
  rectangle "==Клиентское приложение\n//[C# .NET 8, WinForms]//\n\nUI.Desktop +
Core.BusinessLogic\n+ Data.ApiClient + Security.Auth" as desktop <<container>>
  rectangle "==Интеграционный шлюз\n//[ASP.NET Core Web API]//\n\nGET /sections/schedule\nPOST
/equipment/rent" as gateway <<container>>
  database "==БД спорткомплекса\n//[PostgreSQL]//\n\nСекции, занятия,\nсклад инвентаря, брони" as
db <<container>>
}

rectangle "==SSO вуза\n\nАутентификация,\nвыдача JWT" as sso <<external_system>>
rectangle "==СКУД кампуса\n\nПроход в спорткомплекс\nпо пропуску" as skud <<external_system>>

student --> desktop : Смотрит расписание,\nбронирует инвентарь
coach --> desktop : Работает с группой
desktop --> gateway : REST/JSON по openapi.yaml\n[HTTPS]
desktop --> sso : Получает токен\n[OAuth 2.0]
gateway --> db : Чтение/запись\n[SQL]
gateway --> skud : Проверка допуска\n[REST]
@enduml
```